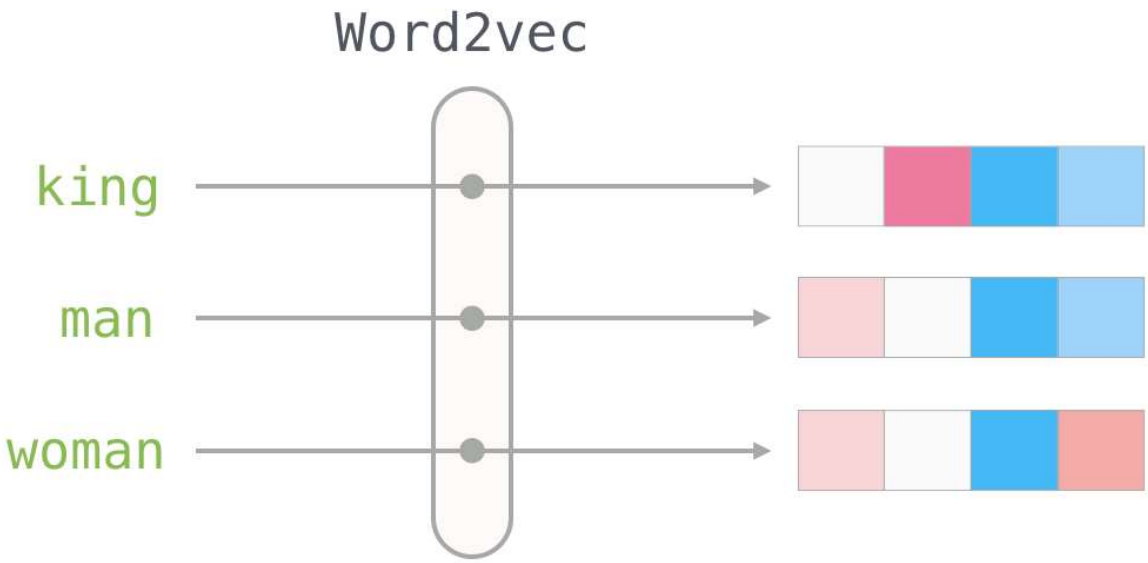
The Illustrated Word2vec

Discussions: Hacker News (347 points, 37 comments) (https://news.ycombinator.com/item?id=19498356), Reddit r/MachineLearning (151 points, 19 comments) (https://www.reddit.com/r/MachineLearning/comments/b60jtg/p_the_illustrated_word2vec/)
Translations: Chinese (Simplified) (https://mp.weixin.qq.com/s?__biz=MjM5MTQzNzU2NA==&mid=2651669277&idx=2&sn=bc8f0590f9e340c1f1359982726c5a30&chksm=bd4c648e8a3bed9817f30c5a512e79fe0cc6fbc58544f97c857
French (https://lbourdois.github.io/blog/nlp/word_embedding/), Korean (https://databreak.netlify.com/2019-04-25-illustrated_word2vec/), Portuguese (https://pessoalex.wordpress.com/2019/03/29/o-word2vec-ilustrado/), Russian (https://habr.com/ru/post/446530/)



> *"**There is in all things a pattern that is part of our universe. It has symmetry, elegance, and grace** - those qualities you find always in that which the true artist captures. You can find it in the turning of the seasons, in the way sand trails along a ridge, in the branch clusters of the creosote bush or the pattern of its leaves.*
>
> *We try to copy these patterns in our lives and our society, seeking the rhythms, the dances, the forms that comfort. Yet, it is possible to see peril in the finding of ultimate perfection. It is clear that the ultimate pattern contains it own fixity. In such perfection, all things move toward death." ~ Dune (1965)*

I find the concept of embeddings to be one of the most fascinating ideas in machine learning. If you've ever used Siri, Google Assistant, Alexa, Google Translate, or even smartphone keyboard with next-word prediction, then chances are you've benefitted from this idea that has become central to Natural Language Processing models. There has been quite a development over the last couple of decades in using embeddings for neural models (Recent developments include contextualized word embeddings leading to cutting-edge models like BERT (https://jalammar.github.io/illustrated-bert/) and GPT2).

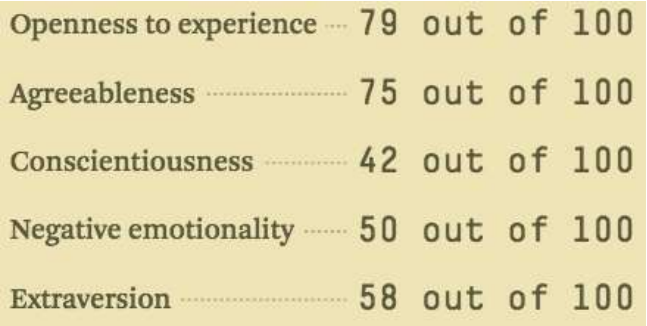
Word2vec is a method to efficiently create word embeddings and has been around since 2013. But in addition to its utility as a word-embedding method, some of its concepts have been shown to be effective in creating recommendation engines and making sense of sequential data even in commercial, non-language tasks. Companies like Airbnb (https://www.kdd.org/kdd2018/accepted-papers/view/real-time-personalization-using-embeddings-for-search-ranking-at-airbnb), Alibaba (https://www.kdd.org/kdd2018/accepted-papers/view/billion-scale-commodity-embedding-for-e-commerce-recommendation-in-alibaba), Spotify (https://www.slideshare.net/AndySloane/machine-learning-spotify-madison-big-data-meetup), and Anghami (https://towardsdatascience.com/using-word2vec-for-music-recommendations-bb9649ac2484) have all benefitted from carving out this brilliant piece of machinery from the world of NLP and using it in production to empower a new breed of recommendation engines.

In this post, we'll go over the concept of embedding, and the mechanics of generating embeddings with word2vec. But let's start with an example to get familiar with using vectors to represent things. Did you know that a list of five numbers (a vector) can represent so much about your personality?

Personality Embeddings: What are you like?

> *"I give you the desert chameleon, whose ability to blend itself into the background tells you all you need to know about the roots of ecology and the foundations of a personal identity" ~Children of Dune*

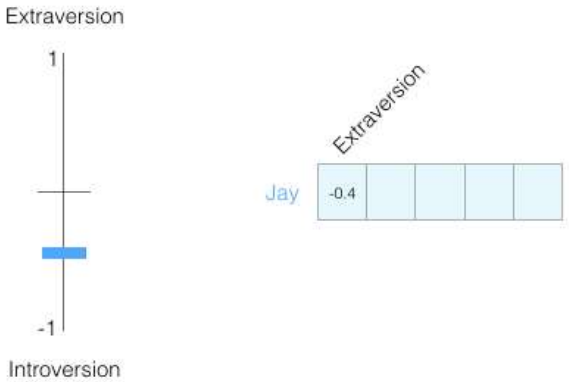
On a scale of 0 to 100, how introverted/extraverted are you (where 0 is the most introverted, and 100 is the most extraverted)? Have you ever taken a personality test like MBTI – or even better, the Big Five Personality Traits (https://en.wikipedia.org/wiki/Big_Five_personality_traits) test? If you haven't, these are tests that ask you a list of questions, then score you on a number of axes, introversion/extraversion being one of them.



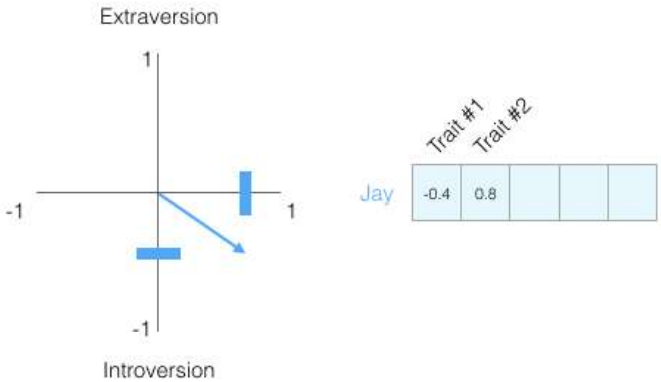
Example of the result of a Big Five Personality Trait test. It can really tell you a lot about yourself and is shown to have predictive ability in academic (http://psychology.okstate.edu/faculty/jgrice/psyc4333/FiveFactor_GPAPaper.pdf), personal (https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1744-6570.1999.tb00174.x), and professional success (https://www.massgeneral.org/psychiatry/assets/published_papers/soldz-1999.pdf). This (https://projects.fivethirtyeight.com/personality-quiz/) is one place to find your results.

Imagine I've scored 38/100 as my introversion/extraversion score. we can plot that in this way:

Let's switch the range to be from -1 to 1:



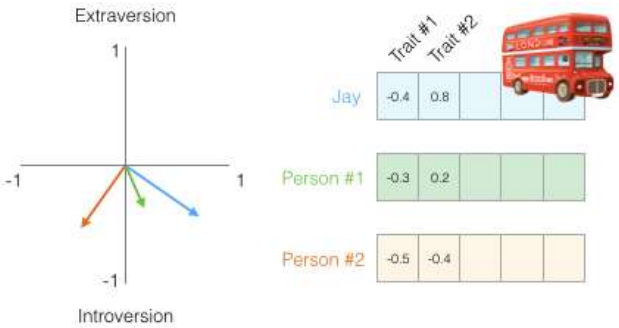
How well do you feel you know a person knowing only this one piece of information about them? Not much. People are complex. So let's add another dimension – the score of one other trait from the test.



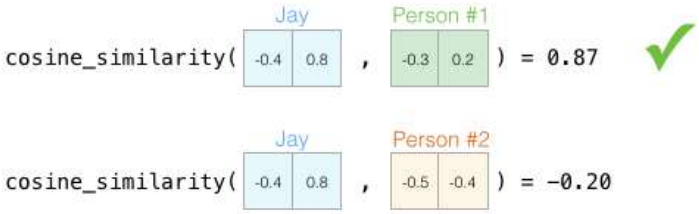
We can represent the two dimensions as a point on the graph, or better yet, as a vector from the origin to that point. We have incredible tools to deal with vectors that will come in handy very shortly.

I've hidden which traits we're plotting just so you get used to not knowing what each dimension represents – but still getting a lot of value from the vector representation of a person's personality.

We can now say that this vector partially represents my personality. The usefulness of such representation comes when you want to compare two other people to me. Say I get hit by a `bus` and I need to be replaced by someone with a similar personality. In the following figure, which of the two people is more similar to me?

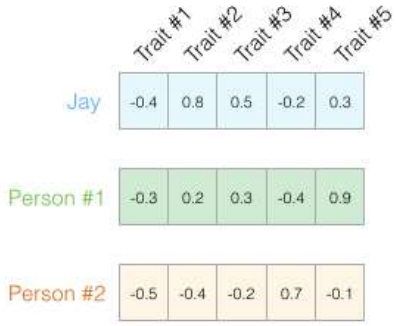


When dealing with vectors, a common way to calculate a similarity score is cosine_similarity (https://en.wikipedia.org/wiki/Cosine_similarity):

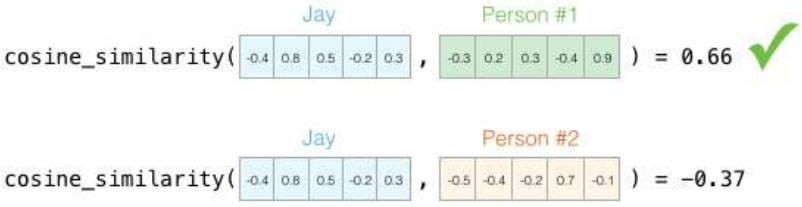


Person #1 is more similar to me in personality. Vectors pointing at the same direction (length plays a role as well) have a higher cosine similarity score.

Yet again, two dimensions aren't enough to capture enough information about how different people are. Decades of psychology research have led to five major traits (and plenty of sub-traits). So let's use all five dimensions in our comparison:



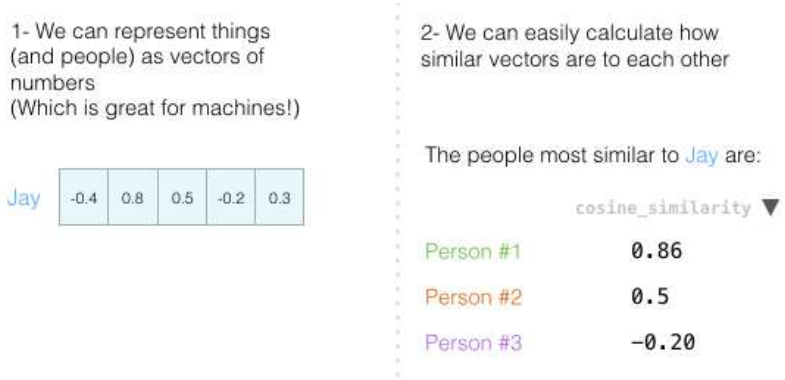
The problem with five dimensions is that we lose the ability to draw neat little arrows in two dimensions. This is a common challenge in machine learning where we often have to think in higher-dimensional space. The good thing is, though, that cosine_similarity still works. It works with any number of dimensions:



cosine_similarity works for any number of dimensions. These are much better scores because they're calculated based on a higher resolution representation of the things being compared.

At the end of this section, I want us to come out with two central ideas:

1. We can represent people (and things) as vectors of numbers (which is great for machines!).
2. We can easily calculate how similar vectors are to each other.



Word Embeddings

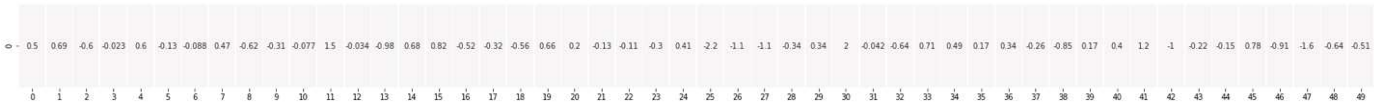
> *"The gift of words is the gift of deception and illusion" ~Children of Dune*

With this understanding, we can proceed to look at trained word-vector examples (also called word embeddings) and start looking at some of their interesting properties.

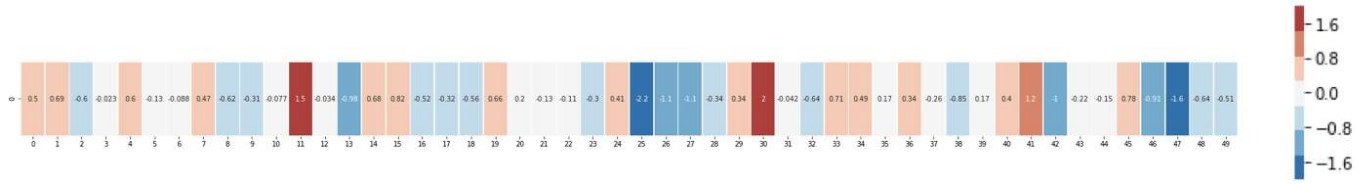
This is a word embedding for the word "king" (GloVe vector trained on Wikipedia):

```
[ 0.50451 , 0.68607 , -0.59517 , -0.022801, 0.60046 , -0.13498 , -0.08813 , 0.47377 , -0.61798 , -0.31012 ,
-0.076666, 1.493 , -0.034189, -0.98173 , 0.68229 , 0.81722 , -0.51874 , -0.31503 , -0.55809 , 0.66421 , 0.1961
, -0.13495 , -0.11476 , -0.30344 , 0.41177 , -2.223 , -1.0756 , -1.0783 , -0.34354 , 0.33505 , 1.9927 ,
-0.04234 , -0.64319 , 0.71125 , 0.49159 , 0.16754 , 0.34344 , -0.25663 , -0.8523 , 0.1661 , 0.40102 , 1.1685 ,
-1.0137 , -0.21585 , -0.15155 , 0.78321 , -0.91241 , -1.6106 , -0.64426 , -0.51042 ]
```

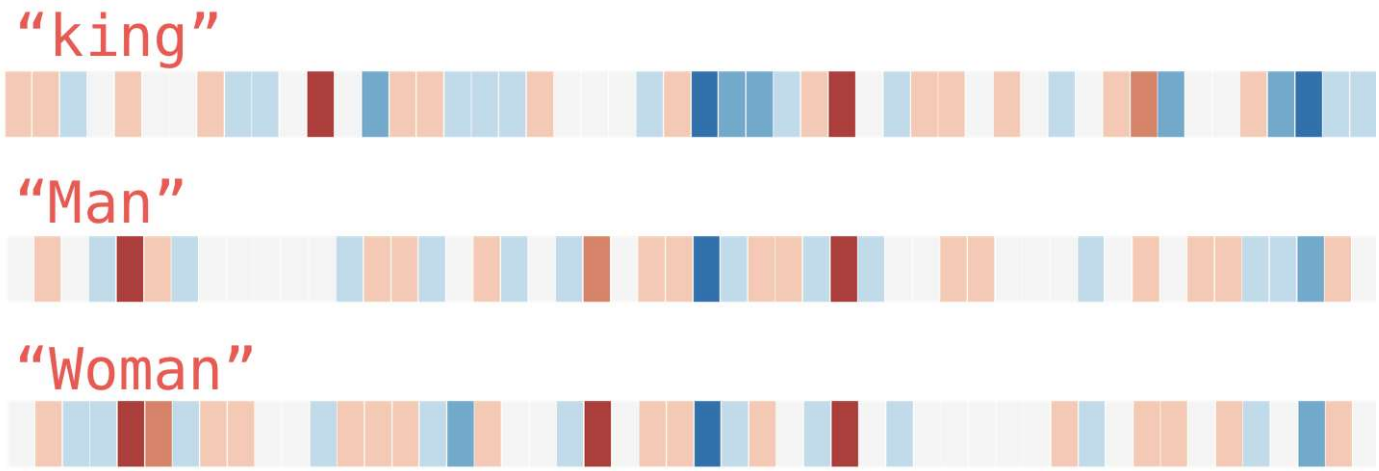
It's a list of 50 numbers. We can't tell much by looking at the values. But let's visualize it a bit so we can compare it other word vectors. Let's put all these numbers in one row:



Let's color code the cells based on their values (red if they're close to 2, white if they're close to 0, blue if they're close to -2):

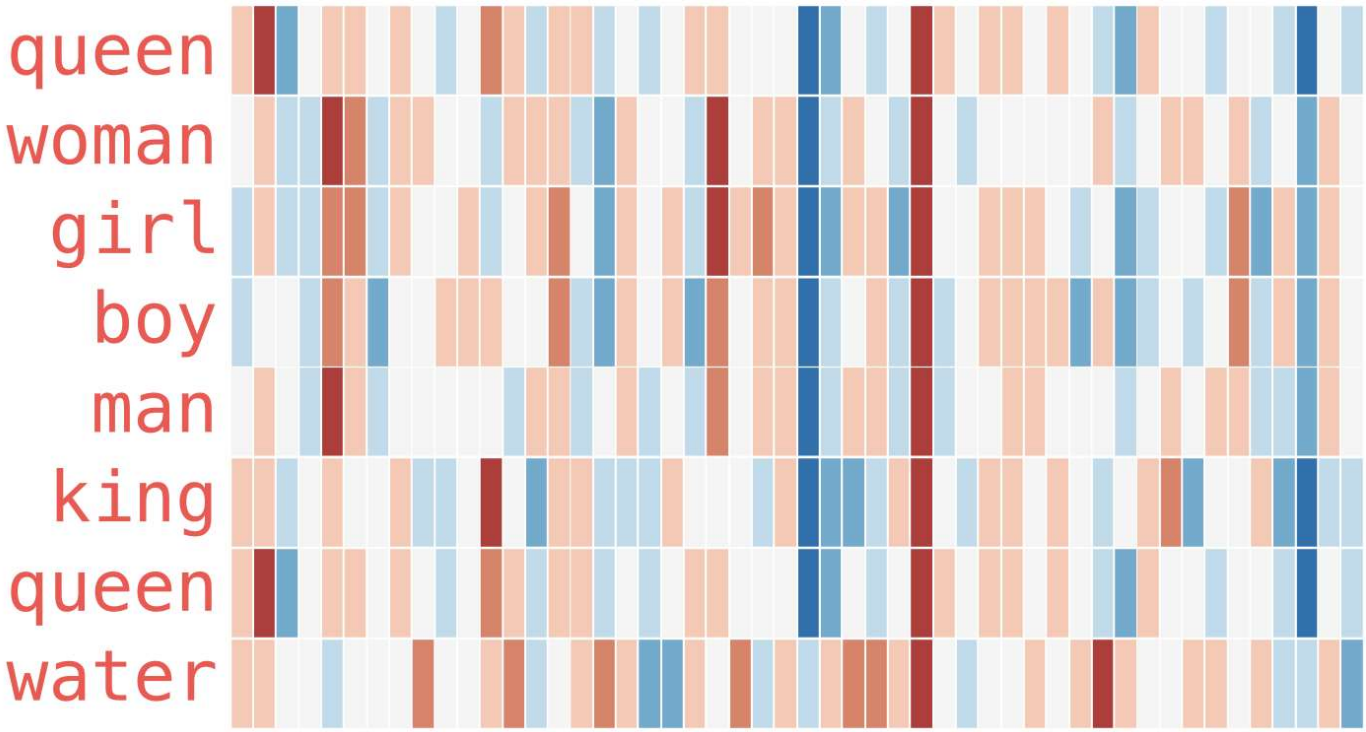


We'll proceed by ignoring the numbers and only looking at the colors to indicate the values of the cells. Let's now contrast "King" against other words:



See how "Man" and "Woman" are much more similar to each other than either of them is to "king"? This tells you something. These vector representations capture quite a bit of the information/meaning/associations of these words.

Here's another list of examples (compare by vertically scanning the columns looking for columns with similar colors):



A few things to point out:

1. There's a straight red column through all of these different words. They're similar along that dimension (and we don't know what each dimensions codes for)
2. You can see how "woman" and "girl" are similar to each other in a lot of places. The same with "man" and "boy"
3. "boy" and "girl" also have places where they are similar to each other, but different from "woman" or "man". Could these be coding for a vague conception of youth? possible.
4. All but the last word are words representing people. I added an object (water) to show the differences between categories. You can, for example, see that blue column going all the way down and stopping before the embedding for "water".
5. There are clear places where "king" and "queen" are similar to each other and distinct from all the others. Could these be coding for a vague concept of royalty?

Analogies

> *"Words can carry any burden we wish. All that's required is agreement and a tradition upon which to build." ~God Emperor of Dune*

The famous examples that show an incredible property of embeddings is the concept of analogies. We can add and subtract word embeddings and arrive at interesting results. The most famous example is the formula: "king" - "man" + "woman":

```
model.most_similar(positive=["king","woman"], negative=["man"])
```

```
[('queen', 0.8523603677749634),
 ('throne', 0.7664333581924438),
 ('prince', 0.7592144012451172),
 ('daughter', 0.7473883032798767),
 ('elizabeth', 0.7460219860076904),
 ('princess', 0.7424570322036743),
 ('kingdom', 0.7337411642074585),
 ('monarch', 0.721449077129364),
 ('eldest', 0.7184862494468689),
 ('widow', 0.7099430561065674)]
```

Using the Gensim (https://radimrehurek.com/gensim/) library in python, we can add and subtract word vectors, and it would find the most similar words to the resulting vector. The image shows a list of the most similar words, each with its cosine similarity.

We can visualize this analogy as we did previously:

The resulting vector from "king-man+woman" doesn't exactly equal "queen", but "queen" is the closest word to it from the 400,000 word embeddings we have in this collection.

Now that we've looked at trained word embeddings, let's learn more about the training process. But before we get to word2vec, we need to look at a conceptual parent of word embeddings: the neural language model.

Class Activity

Featurized Representation of Word Embeddings

	Man	Woman	King	Queen	Apple	Orange
Gender	-1	1	-0.95	0.95	0	0.01
Royal	0.01	0.02	0.93	0.95	-0.01	0
Age	0.03	0.02	0.7	0.69	0.03	-0.02
Food	0.04	0.01	0.02	0.01	0.95	0.97

Embeddings are learned/stored as matrix E